

RESUMEN SISTEMA ANÁLISIS VOLEY — ESTADO COMPLETO

Fecha: 03/06/2026 | Para continuar en nuevo chat sin perder nada

1. REPOSITARIOS

Equipo	GitHub	Vercel
CASLA	github.com/ignacioverdi/Voley-Stats	voley-stats-iota.vercel.app
NAFELS	github.com/ignacioverdi/Voley-Nafels	volley-nafels.vercel.app

2. PARSER DVW — REGLAS CRÍTICAS

Prefijos de línea en SCOUT

- `[*]` = equipo LOCAL (HOME)
- `[a]` = equipo VISITANTE (AWAY)
- Detectar cuál es nuestro equipo comparando nombre en `[3TEAMS]`

Zonas por skill (regla del tilde)

```
ATAQUE  (A): combo = tilde[0],  zones = tilde[1][0:2]
SAQUE   (S): combo = '',        zones = tilde[3][0:2]  ← CRÍTICO
RECEP   (R): combo = '',        zones = tilde[3][0:2]  ← CRÍTICO
SET      (E): call  = tilde[0][:2], zones = tilde[1][0:2]
```

Setter position

- `sc[9]` = setter_pos equipo HOME
- `sc[10]` = setter_pos equipo AWAY
- Set number = `sc[8]`

SO vs TR (Side Out vs Transición)

- Rival sirve → **SO** (atype=0) = nuestro equipo recibe y ataca
- Nosotros servimos → **TR** (atype=1) = transición

3. RECEPCIÓN — DEFINICIÓN DE ZONAS (CRÍTICO)

Sección acumulada (renderRecepcionRival)

Para saber DESDE DÓNDE viene el saque que recibe el jugador:

- **desde_z1** = `srv_orig` del rally in [1, 9] (sacador en zona 1 o zona 6-1)
- **desde_z6** = `srv_orig` in [6]
- **desde_z5** = `srv_orig` in [5, 7] (sacador en zona 5 o zona 6-5)

IMPORTANTE: `srv_orig` se obtiene trackeando la línea de SAQUE anterior en el rally — NO del campo `orig` de la línea de recepción. Porque cuando el sacador está en zona 9, la recepción se codifica con `orig=1` en el DVW.

Dónde cae la recepción (p1/p5/p6)

- **p1** = `dest` in [1, 2, 9] (sector derecho)
- **p6** = `dest` in [3, 6, 8] (sector centro)
- **p5** = `dest` in [4, 5, 7] (sector izquierdo)

Orden visual en pantalla

Z5 - Z6 - Z1 (izquierda → derecha, igual que cancha real)

4. ARMADOR — LÓGICA COMPLETA

Filtro base

Solo se cuentan armadas donde la **recepción inmediatamente anterior** al set fue (#) o (+).

CRÍTICO: "inmediatamente anterior" = el último skill de Amriswil antes del E fue una R. Si entre R y E hay otras acciones (bloqueo, defensa, ataque previo), `rec_quality = '?'` (excluida).

Mapeo combo → zona de ataque

python

```

X5, V5          → Z4  (OH izquierda)
X1, XM, X2, XG,
XC, XD, X7      → Z3  (MB centro)
X6, V6          → Z2  (OPP derecha delantera)
X8, V8 + orig 9 → Z9  (OPP fondo, cuando armador delantero P4/P3/P2)
X8, V8 + orig 2 → Z2  (OPP delantera)
XP, XR, XT, X9  → atk_orig real (pipe/back, generalmente Z8)
XB              → atk_orig real
PP (setter dump) → Z2 si armador delantero (P4/P3/P2), excluido si atrás
X3, X4          → Z2

```

Zonas en la canchita del armador

```

RED
Z4  Z3  Z2  ← fila delantera  (y=0.28 en canvas)
Z7  Z8  Z9  ← fila media/fondo (y=0.70 en canvas)

```

Loop de dibujo: `[4,3,2,7,8,9]`

Orden de canchitas (rotaciones)

```

P4  P3  P2  ← armador delantero (fila superior)
P5  P6  P1  ← armador atrás      (fila inferior)

```

Valores verificados P1 (referencia)

Zona	Armadas	Puntos
Z4	13	10
Z3	12	6
Z2	14	9
Z8	4	1

5. ARCHIVOS CLAVE EN OUTPUTS

Archivo	Descripción
<code>game_plan_amriswil.html</code>	Game Plan Amriswil — TERMINADO y verificado
<code>gp_template.html</code>	Template base para cualquier rival (= gp_lomas.html)

Archivo	Descripción
<code>gp_lomas.html</code>	Referencia CASLA — motor visual completo
<code>amriswil_data_v2.json</code>	Datos Amriswil: attacks/serves/recs por jugador
<code>amriswil_rec_v3.json</code>	Recepción Amriswil con srv_orig del rally correcto
<code>amriswil_setter_rallies_v4.json</code>	Rallies del armador Jovanovic #11 linkeados
<code>amriswil_jugadores.json</code>	JUGADORES data: ataques/saques/recepcion por jugador
<code>nafels_data_v2.json</code>	Datos Nafels: 26 partidos NLA 2025/26
<code>casla_all_actions_v3.json</code>	Datos CASLA: 7 partidos DHM 2026

6. HEATMAPS — ARCHIVOS EN REPO NAFELS

Archivo	Descripción	Estado
<code>ataque_nafels.html</code>	Heatmap ataque Nafels	✓ Subido
<code>saque_nafels.html</code>	Heatmap saque Nafels	✓ Subido
<code>recepcion_nafels.html</code>	Heatmap recepción Nafels	✓ Subido
<code>armador_nafels.html</code>	Heatmap armador Nafels	✓ Subido
<code>ataque_casla.html</code>	Heatmap ataque CASLA	✓ Subido
<code>saque_casla.html</code>	Heatmap saque CASLA	✓ Subido
<code>recepcion_casla.html</code>	Heatmap recepción CASLA	✓ Subido
<code>armador_casla.html</code>	Heatmap armador CASLA	✓ Subido

Formato RAW heatmap ataque

`[rival_idx, gi, set_num, moment, atype, combo_idx, res_idx, orig, dest, rq, rv]` — 11 campos, `r[4]=atype SO/TR`

Formato RAW heatmap saque/recepción

`[rival_idx, gi, set_num, moment, stype_idx, res_idx, orig, dest]` — 8 campos, `stypes` per player

Formato RAW heatmap armador

RAW.setters con 12 campos

[rival,gi,set,mom,atype,call_idx,sp,rq,atk_combo,atk_res,atk_dest,atk_pos]

7. PLANTEL AMRISWIL

#	Nombre	Posición
1	Ramon Diem	LIBERO
6	Bjorn Hohne	PUNTA (OH)
7	Iliya Goldrin	PUNTA (OH)
8	Etienne Schalch	OPUESTO (OPP)
9	Joel Hauck	CENTRAL (MB)
10	Daniel Uruena	CENTRAL (MB)
11	Milan Jovanovic	ARMADOR (S)
14	Bruno SunaricJukic	PUNTA (OH)

8. PLANTEL NAFELS

#	Nombre	Posición
1	Linus Deecke	LIBERO
2	Mathias Corzo	PUNTA
3	Tom Schwitter	PUNTA
4	Ezequiel Vazquez	ARMADOR
5	Joachim Hesselholt	CENTRAL
6	Pablo Denis Cabanas	OPUESTO
7	Roy Schmid	CENTRAL
8	Jonas Peter	LIBERO

#	Nombre	Posición
9	Nathan Broch	PUNTA
10	Dejan Bogdanovski	CENTRAL
11	Christian Bartholet	PUNTA
14	Manuel Figueiredo	OPUESTO
15	Risto Nikolov	CENTRAL

9. GAME PLAN — ESTRUCTURA Y FIXES APLICADOS

Base: `gp_lomas.html` (referencia con todo el motor visual)

Fixes que SIEMPRE hay que aplicar al generar un nuevo GP

1. `item.pts` → `item.pts_pct` en badges (muestra % kills no cantidad)
2. `d.pts+' puntos'` → `d.pts+' kills'` en canvas
3. Loop armador: `[4,3,2,6,1]` → `[4,3,2,7,8,9]`
4. Zona order recepción: `[z1,z6,z5]` → `[z5,z6,z1]` en AMBAS funciones
5. p1/p5/p6 display: orden `[p1,p6,p5]` → `[p5,p6,p1]`
6. Filtro recepción rival: $\text{tot} \geq 15$ acciones (no solo chequear que exista flotado)
7. ARMADOR_DATA via markers `/*__ARM_START__*/` y `/*__ARM_END__*/`

Secciones del Game Plan

- **Tab ATAQUE:** canchitas por combinación por jugador (filtradas por posición)
- **Tab SAQUE:** canchitas SQ/SM por jugador
- **Tab RECEPCIÓN:** flotado/potencia × desde_z1/z5/z6 × p1/p5/p6 (individual)
- **Sección intermedia recepción rival:** jugadores con ≥ 15 recepciones, orden Z5-Z6-Z1
- **Sección armador:** 6 canchitas P4/P3/P2 arriba, P5/P6/P1 abajo + llamadas + EFF por rec quality
- **"Ver heatmap →":** link en cada jugador abre heatmap del rival con player filtrado

10. PRÓXIMOS PASOS PENDIENTES

1. **Heatmaps de Amriswil** — generar `ataque_amriswil.html`, `saque_amriswil.html`, `recepcion_amriswil.html`, `armador_amriswil.html` usando el mismo sistema que Nafels/CASLA pero con los DVW de Amriswil

- 2. **Automatizar GP builder** — el script `gp_builder.py` está en outputs, para nuevo rival solo hay que subir los DVW y correrlo
- 3. **GP para otros rivales** — mismo proceso: subir DVW del rival, correr builder, verificar números vs DataVolley

11. CÓMO GENERAR UN NUEVO GAME PLAN

```
python

# En nuevo chat, con los DVW del rival en /home/claude/DVW_RIVAL/
# y los archivos de outputs disponibles:

build_game_plan(
    dvw_dir      = "/home/claude/DVW_LAUSANNE/",
    rival_name   = "LAUSANNE",
    output_path  = "game_plan_lausanne.html",
    template_path = "gp_template.html", # = gp_lomas.html
    known_positions = {7:"OH", 8:"OPP", 10:"MB", 11:"S", 1:"L"},
    n_games_label = "NLA Suiza 2025/26 · 4 partidos"
)
```

El `gp_builder.py` en outputs tiene toda la lógica encapsulada con los fixes aplicados.

12. VERIFICACIÓN FINAL GAME PLAN AMRISWIL

Todos los valores del armador verificados contra DataVolley:

Rot	Z4	Z3	Z2	Z9	Z8
P1	13/10	12/6	14/9	—	4/1
P6	9/3	13/9	16/4	—	2/0
P5	10/4	19/17	14/7	—	4/2
P4	11/7	14/7	—	7/5	1/1
P3	13/7	9/4	—	7/3	1/0
P2	11/7	8/5	—	11/7	4/3

formato: armadas/kills

13. HEATMAPS DE AMRISWIL — COMPLETADOS

Los 4 heatmaps del rival Amriswil están generados y listos para subir al repo:

Archivo	Descripción
<code>ataque_amriswil.html</code>	Heatmap ataque Amriswil (596 ataques, 9 jugadores)
<code>saque_amriswil.html</code>	Heatmap saque Amriswil (518 saques, 9 jugadores)
<code>recepcion_amriswil.html</code>	Heatmap recepción Amriswil (402 recepciones)
<code>armador_amriswil.html</code>	Heatmap armador Amriswil - Jovanovic #11 (507 armadas)

Links en el GP

- Cada jugador tiene "Ver heatmap →" → abre `recepcion_amriswil.html?player=NUM`
- El armador tiene "Ver heatmap armador →" → abre `armador_amriswil.html`

Para NUEVO RIVAL: mismo proceso

1. Subir DVW del rival al chat
2. Parsear con el mismo parser (detecta automáticamente home/away)
3. Construir los 4 RAW files (atk/srv/rec/set)
4. Inyectar en las 4 plantillas CASLA con sustitución de texto

14. ESTADO FINAL — TODO LO QUE HAY QUE SUBIR AL REPO NAFELS

```
Voley-Nafels repo:
├─ game_plan_amriswil.html    ← NUEVO ✓
├─ ataque_amriswil.html      ← NUEVO ✓
├─ saque_amriswil.html       ← NUEVO ✓
├─ recepcion_amriswil.html   ← NUEVO ✓
├─ armador_amriswil.html     ← NUEVO ✓
├─ ataque_nafels.html        ← actualizado
├─ saque_nafels.html         ← actualizado
├─ recepcion_nafels.html     ← actualizado
├─ armador_nafels.html       ← actualizado
├─ datos_historial.js        ← actualizado
├─ datos_partidos.js         ← actualizado
└─ datos_equipo.js           ← actualizado

Voley-Stats (CASLA) repo:
```


─ ataque_casla.html ← actualizado
─ saque_casla.html ← actualizado
─ recepcion_casla.html ← actualizado
─ armador_casla.html ← actualizado

15. BASE DE DATOS LIGA COMPLETA + AUTOMATIZACIÓN (FINAL)

Base de datos centralizada

`nla_players_db.json` (14MB) — toda la NLA Suiza 2025/26:

- **8 equipos NLA** + 5 europeos
- ~115 jugadores con TODAS sus acciones (atk/srv/rec/sets/blk)
- Cada acción tiene: `rival`, `temporada`, `date`, `atype` (SO/TR), zonas
- 66.177 acciones + 8.958 bloqueos

Fórmulas EFF confirmadas (usar en TODA la app)

```
EFF ataque    = (# - / - =) / total × 100  
EFF saque     = (# + 0.5×/ + 0.25×+ - =) / total × 100  
EFF recepción = (# + 0.5×+ - 0.5×/ - =) / total × 100  
EFF bloqueo   = (# + +) / total × 100
```

Ranking equipos ATK EFF (2025/26)

Pos	Equipo	ATK	SRV	REC	BLK
1	Amriswil	+38.4%	-1.7%	+29.9%	39.9%
2	Schonenwerd	+32.1%	-3.7%	+28.7%	42.7%
3	Nafels	+31.9%	-2.2%	+25.9%	43.6%
4	Lausanne	+30.8%	-11.4%	+24.8%	39.4%
5	Chenois	+27.2%	-3.8%	+26.6%	41.2%
6	Jona	+27.0%	-5.8%	+25.1%	45.4%
7	Colombier	+25.1%	-7.2%	+27.1%	34.7%
8	St Gallen	+21.0%	-8.0%	+17.8%	37.9%

Tabla estadística interactiva

`nla_stats_table.html` — 2 tabs:

- **Ranking Equipos:** cards con EFF de las 4 destrezas + totales
- **Jugadores:** tabla ordenable con todas las métricas (ATK/SRV/REC/BLK desglosadas)
- Filtros: equipo, temporada, posición, búsqueda nombre, mín. ataques
- Click en cualquier columna para ordenar

SISTEMA AUTOMÁTICO — `update_db.py`

Script que hace TODO solo, sin pasar por el asistente:

```
bash
```

```
python3 update_db.py --dvw_dir /ruta/nuevos_dvw/ --temporada 2026/27
```

Pasos automáticos:

1. Parsea los DVW nuevos (detecta home/away solo)
2. Agrega a `nla_players_db.json` SIN borrar temporadas anteriores
3. Etiqueta cada acción con su temporada
4. Regenera `nla_full_stats.json`
5. Regenera `nla_stats_table.html`
6. Regenera los heatmaps de cada equipo

Parámetros:

- `--dvw_dir`: carpeta con DVW nuevos (obligatorio)
- `--temporada`: etiqueta de temporada, ej "2026/27" (default 2025/26)
- `--db_path`: ruta de la BD (default `nla_players_db.json`)
- `--output_dir`: dónde guardar los HTML (default ".")
- `--template_dir`: dónde están los `casla_src_*.html` (default ".")
- `--filter_temporada`: mostrar solo una temporada en stats (default: la que subís)

Detecta archivos ya procesados (por nombre) y los saltea — no duplica.

Filtro de temporada

La tabla y los heatmaps tienen filtro de temporada. Cuando subas DVW de 2026/27, podés elegir ver solo 2025/26, solo 2026/27, o todas juntas. Cada jugador acumula sus datos por temporada sin pisar los anteriores.

Archivos del sistema automático (en outputs)

Archivo	Función
update_db.py	Script maestro de automatización
nla_stats_template.html	Template con placeholders para regenerar tabla
nla_players_db.json	Base de datos principal (14MB)
nla_full_stats.json	Estadísticas calculadas
nla_stats_table.html	Tabla interactiva generada
casla_src_ataque_casla.html	Template heatmap ataque
casla_src_saque_casla.html	Template heatmap saque
casla_src_recepcion_casla.html	Template heatmap recepción

Workflow futuro completo

```
1. Termina la jornada → tenés DVW nuevos
2. Los ponés en una carpeta
3. python3 update_db.py --dvw_dir ./jornada_5/ --temporada 2026/27
4. Se actualiza la BD + tabla + heatmaps automáticamente
5. Subís los archivos generados al repo
6. Para un GP: gp_builder.py lee de la BD actualizada
```